

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Technology (CSE)

QUIZ 2**DURATION: 20 MINUTES****SUMMER SEMESTER, 2024-2025****FULL MARKS: 15**

SWE 4201: Object Oriented Concepts I

Programmable calculators are not allowed.

Answer **all 4 (four)** questions. Figures in the right margin indicate full marks of questions whereas corresponding CO and PO are written within parentheses.

1. What are class invariants? How can encapsulation help maintain class invariants?

2.5
(CO1)
(PO1)

Solution:

A **class invariant** is a condition that must always remain true for an object whenever it is in a valid state.

For example, in a bank account class, the balance should never be negative.

Encapsulation helps maintain class invariants by preventing direct access to data members. Instead, data is accessed and modified through methods (such as constructors, getters, and setters), where validation rules can be enforced to ensure the object remains in a valid state.

2. What advantages does compile-time polymorphism provide in software development?

2.5
(CO1)
(PO1)

Solution:

Compile-time polymorphism allows multiple methods with the same name but different parameter lists. The compiler determines which method to call during compilation.

Its benefits include:

- **Readability:** Programs become more intuitive because the same method name can be used for similar operations. For example, `add()` always represents an addition operation regardless of the input type.
- **Flexibility:** Different implementations of the same method can handle different types or numbers of inputs.
- **Reduced Complexity:** There is no need to create separate method names for similar operations, making the code easier to write and maintain.

3. The following program contains **four compile-time/logical errors**. Identify the incorrect lines and rewrite **only the corrected lines**.

4
(CO2)
(PO2)

```
1 class Vehicle {  
2  
3     private String brand;  
4  
5     public Vehicle(String brand) {  
6         this.brand = brand;  
7     }
```

```

8
9     public String getBrand() {
10         return this.brand;
11     }
12 }
13
14 class Car extends Vehicle {
15
16     private int seats;
17
18     public Car(String brand, int seats) {
19         seats = seats;
20     }
21
22     public int getSeats() {
23         return getBrand();
24     }
25
26     public void display() {
27         System.out.println(brand + " " + seats);
28     }
29 }

```

Solution:

Error 1

Incorrect:

```

1 public Car(String brand, int seats) {
2     seats = seats;
3 }

```

Correct:

```

1 public Car(String brand, int seats) {
2     super(brand);
3     this.seats = seats;
4 }

```

Error 2

Incorrect:

```

1 seats = seats;

```

Correct:

```

1 this.seats = seats;

```

Error 3

Incorrect:

```

1 return getBrand();

```

Correct:

```
1 return seats;
```

Error 4

Incorrect:

```
1 System.out.println(brand + " " + seats);
```

Correct:

```
1 System.out.println(getBrand() + " " + seats);
```

4. Complete the `Course` class so that the following program executes successfully and produces the expected output. You may add attributes, constructors and methods but you may **not** modify the `main()` method. 6
(CO2)
(PO2)

```
1 public class Main {  
2  
3     public static void main(String[] args) {  
4  
5         Course c = new Course("SWE4201");  
6  
7         c.addStudent("Alice");  
8         c.addStudent("Bob");  
9         c.addStudent("Alice");  
10  
11         System.out.println(c.getCode());  
12         System.out.println(c.getStudentCount());  
13     }  
14 }
```

Expected Output:

```
1 SWE4201  
2 2
```

Requirements:

1. code and students must be private.
2. Use an `ArrayList<String>` to store student names.
3. Duplicate student names should not be added.
4. Implement any necessary constructor(s) and method(s).

Solution:

```
1 import java.util.ArrayList;  
2  
3 class Course {  
4  
5     private String code;  
6     private ArrayList<String> students;  
7  
8     public Course(String code) {
```

```
9      this.code = code;
10     students = new ArrayList<>();
11 }
12
13 public void addStudent(String name) {
14
15     if (!students.contains(name)) {
16         students.add(name);
17     }
18 }
19
20 public String getCode() {
21     return code;
22 }
23
24 public int getStudentCount() {
25     return students.size();
26 }
27 }
```

Rubric:

- Private attributes (code and students) – 1 mark
- Correct ArrayList<String> declaration and initialization – 1 mark
- Constructor implementation – 1 mark
- Duplicate prevention using contains() – 1.5 marks
- addStudent() method implementation – 0.5 mark
- getCode() method – 0.5 mark
- getStudentCount() method – 0.5 mark